TRIP PLANNER PWA · FICHE MODULE · MEMBRE 5

Module 5 — Gestion de la checklist

Ce que fait le module, la logique métier qu'il porte, et ce que vous devez savoir présenter.

COLLECTION MONGODB	FONCTIONNALITÉ AVANCÉE	RÔLE TRANSVERSE
checklist_items (+ checklist_templates)	**Checklist générée à partir des préférences du voyage**	—

1. Objectif du module

Produire automatiquement une liste de bagages et de préparatifs adaptée au voyage (type, activités, hébergement, transport, voyageurs, besoins spécifiques, saison, durée, destination), puis laisser l'utilisateur la compléter, la modifier et cocher ce qui est prêt. Les règles sont des données (collection de modèles), pas du code figé.

> Règle d'équipe : vous êtes le seul à écrire dans votre collection. Les autres modules passent par votre API (ou votre service), et toute modification d'un modèle partagé demande l'accord du groupe.

2. Modèle de données

Champ	Type	Règle / rôle
`checklist_items.trip`	ref trips	obligatoire
`label`	texte	1 à 120 caractères
`quantity`	entier	1 à 999
`category`	enum	documents, clothing, toiletries, health, electronics, gear, baby, money, other
`isPacked`	booléen	coché = dans la valise
`source`	generated | manual	origine de l'article
`templateKey`	texte | null	règle qui a produit l'article
`isEdited`	booléen	article généré modifié par l'utilisateur
`checklist_templates.key`	texte unique	identifiant de la règle
`condition`	objet	condition à évaluer (voir ci-dessous)
`items[]`	objets	{ label, category, isCountable }

3. Logique métier

Contexte calculé pour chaque voyage

- Les **préférences** saisies à la création : tripType, activities, accommodation, transport, travellers, specialNeeds.
- **season** (dérivé) : mois de départ + hémisphère du pays de destination. Hiver = décembre à février au nord ; les saisons sont inversées pour les pays de l'hémisphère sud.
- **duration** (dérivé) : nombre de jours calendaires, départ et retour inclus.

- **isAbroad** : pays de destination ≠ pays de résidence du propriétaire (si le pays de résidence est inconnu, on considère le voyage à l'étranger).

Moteur de règles (data-driven)

- Chaque modèle contient une condition et une liste d'articles ; il « se déclenche » si la condition est vraie.
- Opérateurs : `always`, `eq`, `neq`, `in`, `includes` (pour les listes), `gt`, `gte`, `lt`, ainsi que les combinaisons `any` (OU) et `all` (ET). Les champs imbriqués sont permis, par exemple `travellers.infants`.
- Exemple : `{ any: [ {field:"tripType", op:"eq", value:"mountain"}, {field:"activities", op:"includes", value:"hiking"} ] }`.
- Les articles sont **dédupliqués par libellé** (sans tenir compte de la casse) : si deux règles ajoutent « Sunscreen », il n'apparaît qu'une fois.
- **Règle de quantité** : un article comptable (sous-vêtements, chaussettes, t-shirts) vaut **min(durée + 1, 10) × nombre d'adultes** ; les autres valent 1.
- 18 modèles sont chargés par `npm run seed` : les 11 du cahier des charges et 7 complémentaires (tenue formelle, conduite à l'étranger, tenue modeste ou pèlerinage, business, ski, natation, été). L'application ne les modifie jamais.

Régénération (la règle importante)

- Elle se déclenche à la **création** du voyage, à chaque **changement de préférences, de dates ou de pays**, et avec le bouton « Regenerate ».
- Elle **ajoute** les articles manquants ; un article ajouté à la main avec le même libellé compte comme présent.
- Elle **met à jour** la quantité et la catégorie des articles générés que l'utilisateur n'a pas touchés.
- Elle **supprime** les articles générés qui ne correspondent plus, **sauf s'ils ont été modifiés ou cochés**.
- Elle ne touche **jamais** aux articles ajoutés à la main.
- Modifier le libellé, la quantité ou la catégorie d'un article généré le marque `isEdited` : il est protégé.
- La génération se fait **côté serveur** : le même voyage donne la même liste sur tous les appareils.

4. Fonctionnalité avancée — Checklist générée à partir des préférences du voyage

Exemple : Crète, 7 jours en juillet, type beach, activité diving, 1 adulte, pays de résidence Tunisie. Résultat : 25 articles.

– règles *Always*, *beach*, *diving*, *plane*, *abroad* et *summer* déclenchées ;

– chaussettes, t-shirts et sous-vêtements ×8 (7 + 1).

Les écrans affichent pour quel contexte la liste a été générée : saison, durée, type, « à l'étranger », activités.

Hors ligne : on peut cocher, ajouter et modifier (file d'attente). La régénération demande une connexion, puisqu'elle est faite par le serveur.

5. API REST (préfixe `/api/v1`, réponse `{ data, message, errors }`)

Méthode	Route	Accès	Rôle
`GET / POST`	`/trips/:tripId/checklist`	membre	articles + contexte / ajouter un article manuel
`PATCH / DELETE`	`/trips/:tripId/checklist/:itemId`	membre	modifier, cocher / supprimer
`POST`	`/trips/:tripId/checklist/generate`	membre	régénérer (ajoutés, mis à jour, supprimés)
`GET`	`/checklist/templates`	membre	consulter les règles

6. Écrans (front Next.js)

- Onglet « Checklist » : progression, contexte de génération, ajout rapide, filtres (tous, à préparer, prêts), groupes par catégorie, badges « auto » et « edited »
- Bouton « Regenerate from preferences » ; fenêtre de modification d'un article

7. Fichiers dont vous êtes responsable

API — trip-planner-api

- `src/modules/checklist/ (checklist.model, generator.ts, checklist.routes)`
- `src/seed/templates.ts, src/seed/seed.ts`
- `src/lib/countries.ts (saison, hémisphère, durée)`

Front — trip-planner-web

- `src/app/trips/[id]/checklist/page.tsx`

8. Liens avec les autres modules

- **Module 2** : préférences, dates, pays ; génération dans la transaction de création du voyage.
- **Module 1** : pays de résidence (`homeCountry`) du propriétaire.

9. À démontrer à la soutenance

- ☐ Créer un voyage beach + diving : montrer la liste générée et les quantités.
- ☐ Cocher un article et en modifier un autre, passer le voyage en « mountain », régénérer : ajouts montagne, et les articles cochés ou modifiés restent.
- ☐ Montrer la collection `checklist_templates` : les règles sont des données.

10. Reste à faire

- **Tests unitaires du moteur de règles** (exigés) : chaque opérateur, saison selon l'hémisphère, règle de quantité, déduplication, et le fait que la régénération ne supprime jamais un article modifié ou coché.
- Question ouverte : API météo pour affiner la saison (reportée).
- Requêtes Postman.

11. Questions probables du jury

Pourquoi des règles en base plutôt qu'en code ?
Pour ajouter ou modifier une règle sans redéployer : c'est l'exigence « data-driven » du cahier des charges.

Comment éviter d'effacer le travail de l'utilisateur ?
Les drapeaux isEdited et isPacked protègent l'article ; les articles manuels ne sont jamais touchés.